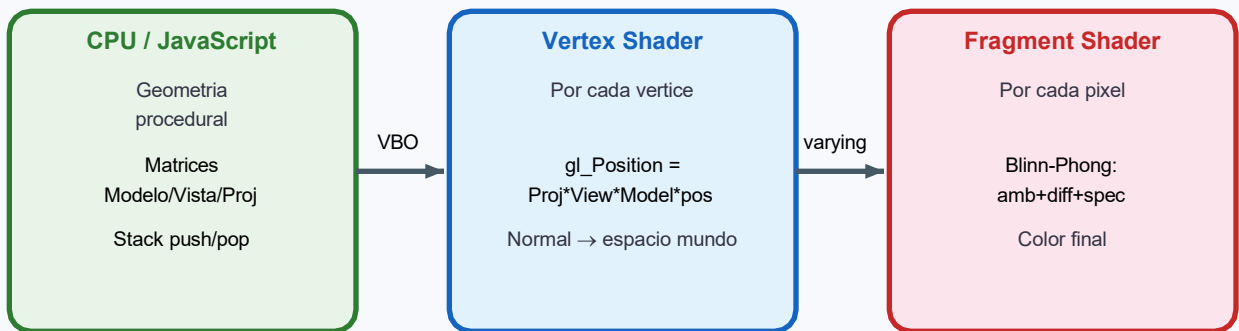


Graficas de Bajo Nivel

WebGL2 • WebGPU+WGSL • WebGPU+TSL

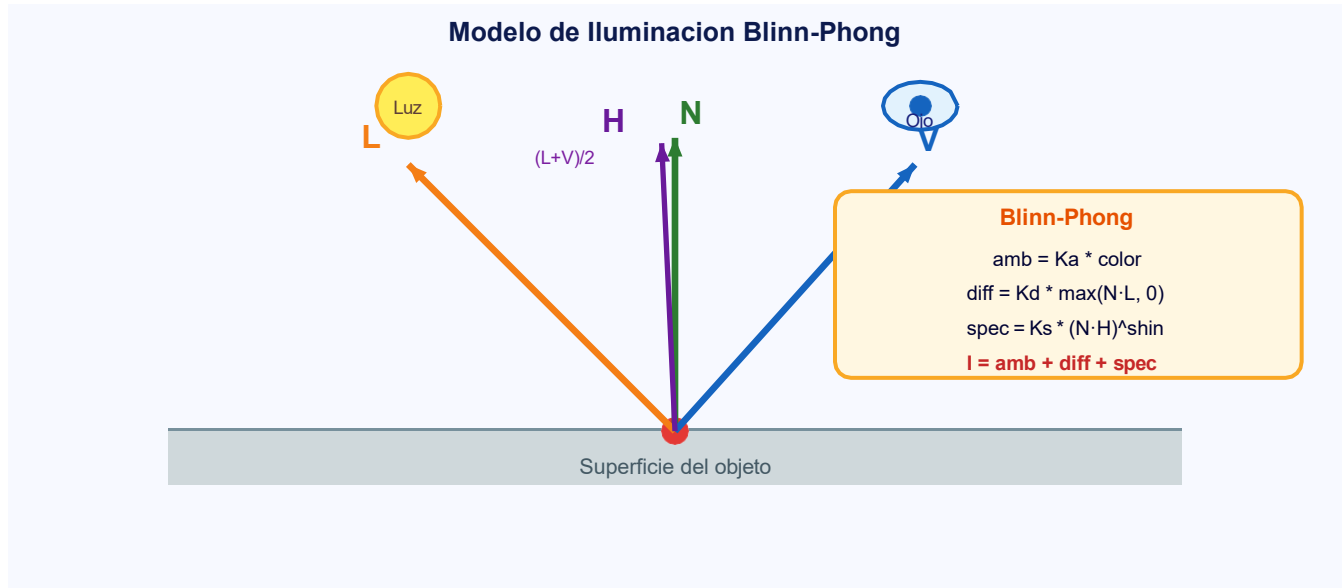
Iluminacion Phong difusa + Blinn-Phong especular
Brazo Robotico & Sistema Solar con jerarquias de matrices

Pipeline de Renderizado (WebGL2 / WebGPU)



Modelo de Iluminacion

Los tres proyectos implementan el mismo modelo de iluminacion local: una componente ambiental constante, difusa lambertiana (Phong) y especular Blinn-Phong. La clave de Blinn-Phong es el **vector half-way H**, que es la bisectriz entre la direccion de la luz (L) y la del observador (V). Usar H en lugar del vector de reflexion R es mas eficiente y produce resultados mas suaves en superficies reales.



Parte A — WebGL2 + GLSL ES 3.0

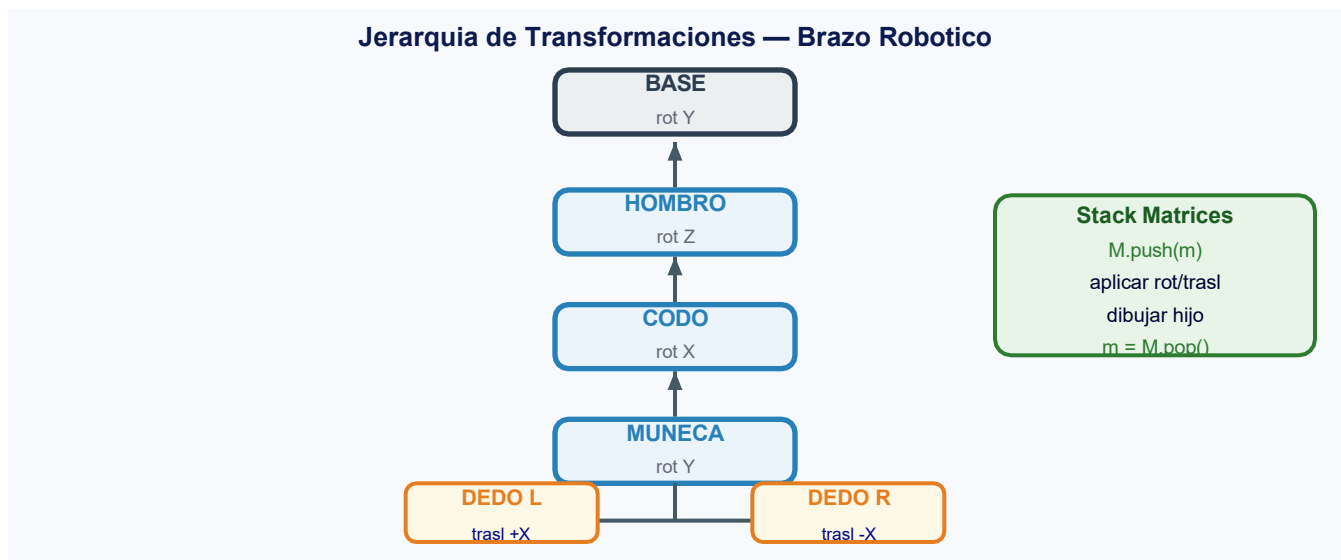
WebGL2 es la API de graficos web basada en OpenGL ES 3.0. Todo se configura manualmente: desde la compilacion de shaders hasta los buffers de vertices. No se usa ningun framework; cada operacion matematica y cada llamada al GPU esta escrita explicitamente.

Matrices completamente manuales

Se implemento una biblioteca propia de algebra lineal con matrices 4x4 para traslacion, rotacion y escala, matriz de perspectiva y lookAt para la camara, y la **matriz normal** (transpuesta de la inversa 3x3) para transformar correctamente las normales cuando hay escala no uniforme.

Stack push/pop para jerarquias

La jerarquia del brazo se implementa con un stack de matrices: antes de entrar a un nodo hijo se guarda la matriz actual (push), se aplica la transformacion local, se dibuja, y al salir se restaura la matriz del padre (pop). Esto permite que cada articulacion opere en su propio espacio local.



VAOs, VBOs y shaders GLSL

Las geometrias (caja, cilindro, esfera) se generan proceduralmente y se suben al GPU en Vertex Buffer Objects. Un Vertex Array Object encapsula la configuracion de atributos para cada malla. Los shaders GLSL calculan la iluminacion Blinn-Phong por fragmento con todas las matrices pasadas como uniforms.

Parte B — WebGPU + WGSL

WebGPU es la API moderna que reemplaza a WebGL. Introduce un modelo de programación más explícito y seguro, inspirado en Vulkan y Metal. Los shaders se escriben en WGSL (WebGPU Shading Language).

Pipeline inmutable

A diferencia de WebGL donde el estado es global y mutable, en WebGPU el **render pipeline** se define una vez y no puede cambiarse. Esto incluye el formato de los vértices, el blend state, el depth-stencil y los shaders. El resultado es que el driver GPU puede optimizar mucho más agresivamente.

Bind Groups y alineamiento de memoria

Los uniforms (matrices, colores, parámetros de luz) se pasan al shader mediante **bind groups** que apuntan a GPU Buffers. WGSL exige que cada campo de un struct esté alineado a 16 bytes, por lo que la matriz normal (mat3x3) se almacena como tres vec4 con padding, ocupando 48 bytes en lugar de 36.

Command Encoder

WebGPU usa un modelo de grabación de comandos: se crea un CommandEncoder, se graban todos los draw calls del frame (setPipeline, setBindGroup, drawIndexed...) y al final se envían en lote con device.queue.submit(). Esto reduce el overhead CPU y mejora la eficiencia del GPU.

Shader en WGSL

La lógica Blinn-Phong es idéntica a la Parte A pero en sintaxis WGSL. WGSL es un lenguaje con tipos explícitos y sin conversión implícita, lo que hace los shaders más predecibles. Los entry points del vertex shader y fragment shader se anotan con @vertex y @fragment.

Parte C — WebGPU + TSL

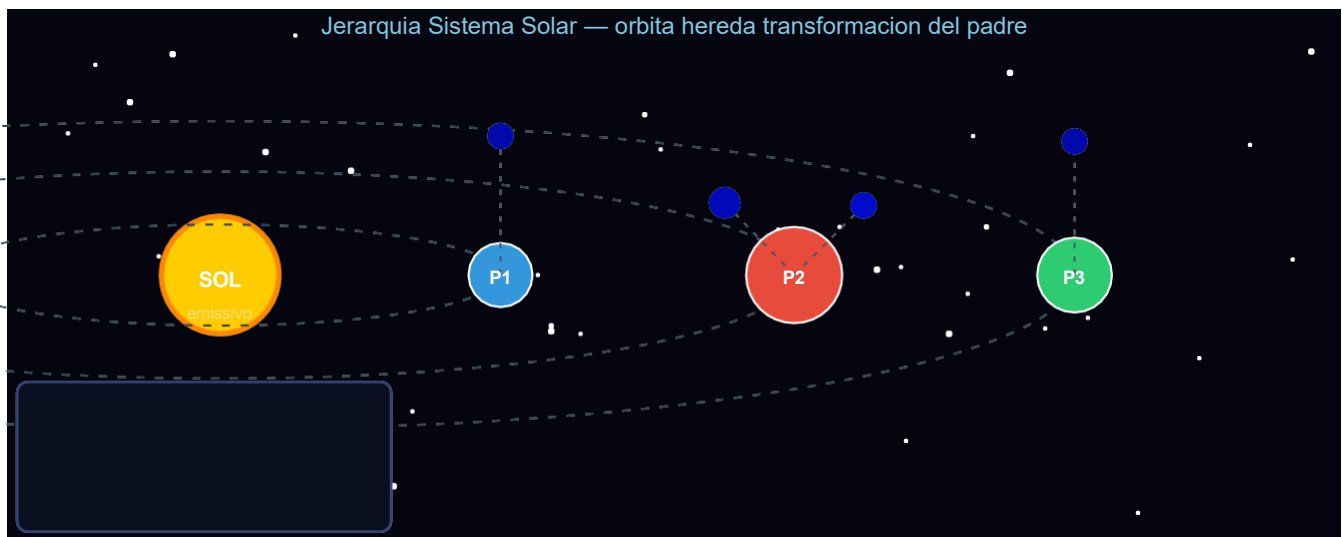
TSL (Three Shading Language) es el sistema de nodos de shading de Three.js a partir de la version r163+. Permite escribir el shader como código JavaScript funcional que Three.js compila automáticamente a WGSL o GLSL según el backend activo. Esto hace los shaders portables sin escribir ninguna línea de WGSL o GLSL manualmente.

Shader como grafo de nodos

Cada operación (normalize, dot, pow, add, mul) es un nodo. Los nodos se combinan para describir el shader de forma declarativa. Nodos built-in como **normalWorld** y **positionWorld** dan acceso directo a la normal y posición en espacio mundo sin necesidad de uniforms extra. El nodo **uniform()** crea un uniform que Three.js sincroniza automáticamente.

Jerarquía con THREE.Group

En esta parte se aprovecha el grafo de escena de Three.js para las jerarquías, mientras que toda la lógica de iluminación es completamente propia (TSL). Cada planeta es hijo de un Group que actúa como pivot de órbita, y cada luna es hija de un Group hijo del planeta, heredando automáticamente su movimiento.



Comparación de las Tres Implementaciones

Cada parte tiene sus ventajas según el caso de uso. WebGL2 tiene la mayor compatibilidad. WebGPU ofrece el máximo control y rendimiento. TSL permite iterar rápidamente en shaders sin conocer el lenguaje destino.

